

1. Introduction:

GovEase AI is an intelligent, privacy-first digital liaison designed to bridge the "Digital Bureaucracy Gap" in Malaysia. As government portals like MyTax, PTPTN, and JPJ MySikap remain fragmented and data-heavy, GovEase AI introduces a multi-agent autonomous workflow that simplifies these complex interactions. By leveraging the **Z AI Model**, the system transforms unstructured user data and physical documents into verified, structured information, allowing for seamless and secure automation of administrative tasks without compromising the user's data sovereignty.

1.1. Purpose:

The primary purpose of this system is to eliminate the repetitive friction associated with Malaysian government digital services. The technical scope centers on three core objectives:

- **Automated Data Orchestration:** Using a multi-agent system (The Concierge, The Guardian, and The Executor) to handle document parsing and form-filling autonomously.
- **Privacy by Design:** Implementing a "local-first" data strategy where sensitive personal information is processed and stored on the user's device rather than a central server.
- **Accuracy and Control:** Ensuring high-fidelity data extraction through AI, while maintaining a mandatory "Human-in-the-Loop" review process to ensure legal and administrative accountability.

1.2. Background:

Despite the rapid digital transformation in Malaysia, the current landscape of public service delivery often requires citizens to manually re-enter identical personal data across multiple siloed platforms. This "manual overhead" leads to user frustration, data entry errors, and security concerns when handling sensitive documents like NRICs or EA forms. Historically, users have relied on basic browser autofill or manual copy-pasting, which lack the semantic intelligence to interpret complex forms or the security measures to protect documents from misuse. **GovEase AI** was born out of the need for a unified "smart layer" that sits between the citizen and the state, utilizing **Agentic AI** to turn a time-consuming bureaucratic chore into a streamlined, conversational experience.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
<i>Malaysian Citizen (End-User)</i>	<i>Engages with the Concierge via natural language or document upload, reviews auto-populated web forms, and authorizes the final submission.</i>	<i>A zero-learning-curve interface, absolute data privacy via local storage, high extraction accuracy, and a drastic reduction in time spent on bureaucracy.</i>
<i>Government Agencies (Passive)</i> <i>(e.g., LHDN, PTPTN, JPJ)</i>	<i>Hosts the destination web portals and processes the final, structured form submissions initiated by the citizen.</i>	<i>Receipt of legitimate, accurately formatted data that complies with existing system requirements, without experiencing automated bot spam.</i>
<i>Development Team</i>	<i>Architects, engineers, and maintains the multi-agent system (Concierge, Guardian, Executor) and optimizes Z AI prompt strategies</i>	<i>Clear documentation on heuristic DOM mapping, manageable token inference costs, and a modular codebase supporting rapid iteration.</i>
<i>QA & Security Team</i>	<i>Audits the system for vulnerabilities, validates AES-256 encryption protocols, and tests AI fallback behaviors (e.g., blurry uploads, portal UI changes).</i>	<i>Defined testing parameters for OCR confidence thresholds, clear error-handling logic, and guaranteed validation of local-first data isolation.</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Progressive Web App (PWA) & Cross-Browser Extension
Architecture	Local-First, Multi-Agent Client-Side Ecosystem

GovEase AI is primarily structured as a secure, decentralized multi-agent system operating directly within the user's browser environment. Instead of a traditional client-server model, the system utilizes three specialized autonomous agents: **The Concierge** (Frontend Chat/Upload UI), **The Guardian** (Local Security & Watermarking), and **The Executor** (DOM-manipulating Browser Extension). Rather than sending personal data to a backend API layer, all data extraction and semantic parsing are powered by the **Z AI Model**, with the resulting structured data encrypted and stored locally in the browser's IndexedDB. The Executor agent then interacts directly with external government web portals to populate fields, ensuring zero-trust data sovereignty.

2.1.2 LLM as Service Layer

In GovEase AI, the **Z AI Model** is not treated as a generic chatbot module; it acts as the central **Semantic and Extraction Service Layer**. Because the system relies on a local-first architecture without a traditional backend database for processing, the Z AI API handles all heavy lifting regarding unstructured data interpretation. It is integrated directly into the **Concierge (PWA)** to process natural language inputs, execute Optical Character Recognition (OCR) on uploaded documents, and map that data into strict JSON schemas before it is encrypted and stored locally.

2.1.3 Dependency Diagram

a. Prompt Construction and API Transmission:

Prompts are constructed dynamically within the Concierge PWA. When a user uploads a document, the system wraps the base64-encoded image in a predefined system prompt template (e.g., "Extract the following fields from this image: [NRIC, Name, Income]. Return strictly as JSON."). This packaged payload is sent securely via HTTPS to the Z AI Model API.

b. Context Window Contents:

The context window sent to the Z AI Model contains:

- 1. System Directives: Strict instructions enforcing the JSON output format and the "Zero-Imputation" rule (returning null if data is missing).
- 2. Target Schema: The specific data fields required by the target government portal (e.g., LHDN tax relief categories).

3. User Payload: The compressed image data (for OCR) or the recent conversational text history (limited to the last 5 turns to manage costs).

c. Receiving, Parsing, and Passing Responses:

Once the Z AI Model returns the response, the Concierge PWA intercepts it.

1. The system parses the response to ensure it perfectly matches the required JSON schema.
2. If validation passes, the JSON payload is passed to the Encryption Module (AES-256).
3. The encrypted structured data is then saved into the browser's IndexedDB, completing the Concierge's workflow and passing the baton to the Executor extension.

d. Token Limitation and Input Chunking:

To strictly enforce LLM token limits and keep inference costs manageable:

- Image Compression: All uploaded documents are compressed client-side to under 5MB before being encoded and sent to the model.
- Chat Pruning: The conversational context window is aggressively pruned; only the immediate instructions and the last 3-5 conversational turns are sent to the API, dropping older, irrelevant greetings.

e. Major API Calls Between Components:

- Concierge PWA $\rightarrow$ Z AI API: Sends constructed prompts and receives JSON-structured data.
- Concierge PWA $\rightarrow$ IndexedDB API: Writes encrypted user data to local browser storage.
- Executor Extension $\rightarrow$ IndexedDB API: Reads and decrypts data when a target portal is detected.

f. Component Interactions (Frontend, Services, Local DB):

Unlike traditional cloud apps, GovEase AI bypasses a centralized backend.

- The Frontend (Concierge) interacts directly with the External API (Z AI Model).
- The data is then committed to the Local Database (IndexedDB).
- Finally, the Executor Extension interacts directly with the External Government Portals (DOM structure). There is no direct backend-to-database communication on external servers, ensuring absolute data privacy.

g. Highlight of Distributed Components:

While not distributed across AWS microservices, GovEase AI is distributed across the user's client environment:

1. The PWA (Concierge): Hosted via a static CDN, acts as the primary user interface and AI orchestrator.
2. The Browser Extension (Executor): Installed locally, acts as the isolated execution environment that monitors active tabs for government URLs.
3. The Cloud Service (Z AI Model): The only external processing node, functioning strictly statelessly to interpret data without retaining it.

2.2.2. Sequence Diagram

Phase 1: Secure Ingestion (Local-First)

1. **Citizen** → **Concierge PWA**: Uploads a photo of an EA Form (salary slip).
2. **Concierge PWA** → **Guardian**: Passes the image for immediate processing.
3. **Guardian [Local Process]**: Applies indelible, client-side visual watermark (e.g., "FOR LHDN TAX USE ONLY") to the image.
4. **Guardian** → **Concierge PWA**: Returns the watermarked, secured image.



Phase 2: AI Processing & Structuring (Local-to-Cloud)

5. **Concierge PWA [Local Process]**: Compresses the watermarked image to meet token cost constraints and wraps it in a dynamically constructed prompt. The prompt mandates strict JSON output schema and prohibits data hallucination (the "Zero-Imputation" rule).
6. **Concierge PWA** → **Z AI Model API**: Transmits the secured, compressed payload via HTTPS for processing.
7. **Z AI Model API [Stateless Cloud Process]**: Performs OCR, interprets Malaysian semantic terms (e.g., mapping "Gaji Pokok" to "Basic Salary"), performs schema validation, and structures the data.
8. **Z AI Model API** → **Concierge PWA**: Returns strictly formatted structured data (JSON).



Phase 3: AES-256 Storage (Local-First)

9. **Concierge PWA** → **Guardian**: Passes raw JSON to be encrypted.
 10. **Guardian [Local Process]**: Prompts the user (via Concierge) for their secret PIN (if not cached), generates the AES-256 encryption key, and encrypts the JSON.
 11. **Guardian** → **Concierge PWA**: Returns encrypted ciphertext.
 12. **Concierge PWA** → **Local Database**: Writes the encrypted structured data to IndexedDB.
- Concierge chat confirms: "Data from EA Form extracted and secured locally."



Phase 4: Agentic Form Filling (Local-to-External Portal)

13. **Citizen** → **External World**: Opens a new tab and navigates to the LHDN MyTax portal.
14. **Executor Extension** https://en.wikipedia.org/wiki/Trigger_%28particle_physics%29: Detects the LHDN MyTax URL and identifies form fields requiring autofill (e.g., ID number, income details).
15. **Executor Extension** → **Citizen**: Prompts for PIN to unlock the localized database.
16. **Citizen** → **Executor Extension**: Provides PIN.
17. **Executor Extension** → **Local Database**: Reads encrypted data.
18. **Local Database** → **Executor Extension**: Returns encrypted data.
19. **Executor Extension** → **Guardian**: Request to decrypt data.
20. **Guardian [Local Process]**: Uses PIN-derived key to decrypt data. Returns raw JSON.
21. **Executor Extension** → **Gov Portal DOM**: Performs heuristic mapping (DOM-aware matching of JSON keys to webpage fields) and populates the form in real-time.



Phase 5: Human-in-the-Loop Interceptor

22. **Executor Extension** → **Citizen**: Signals form completion. The Executor programmatically blocks the AI from clicking the final "Submit" button and triggers a chat notification: *"Review fields on screen."*

23. **Citizen** → **Gov Portal DOM**: Visually inspects the auto-populated data on the live webpage. Citizen corrects one minor discrepancy manually (e.g., clarifying a blurry figure) directly on the portal.

24. **Citizen [Final Verification]**: The Citizen, now responsible for the legal accuracy of the form, manually clicks the real "Submit" button on the government portal, completing the filing process.

2.2. Technological Stack

2.2.1. Frontend (Concierge PWA): React / Next.js with Tailwind CSS – *A Progressive Web App providing a responsive, cross-platform chat interface for mobile and desktop users without requiring app store installation.*

2.2.2. Browser Extension (Executor Agent): React + Plasmo Framework – *Used to build the cross-browser extension responsible for injecting the UI overlay and manipulating the DOM on external government portals.*

2.2.3. AI & Service Layer: Z AI Model REST API – *Acts as the stateless cognitive engine for natural language processing, OCR, and mapping unstructured images to rigid JSON schemas.*

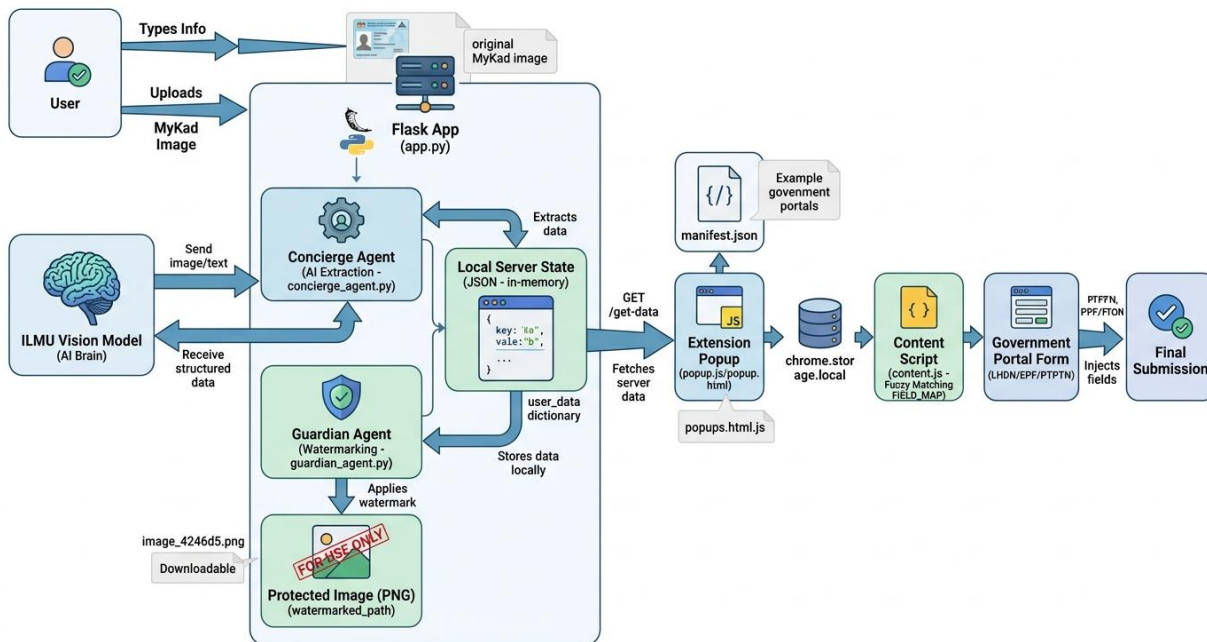
2.2.4. Security (Guardian Agent): Web Crypto API & HTML5 Canvas API – *Native browser APIs used for AES-256 local encryption and client-side image watermarking prior to any network transmission.*

2.2.5. Database: IndexedDB (via Dexie.js) – *An in-browser NoSQL database used to store encrypted user data locally, replacing the need for a traditional cloud relational database (like PostgreSQL) to protect sensitive information.*

2.2.6. Cloud/Deployment: Vercel (for PWA hosting) & Google Chrome Web Store (for Extension distribution).

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

To transition your system to a fully normalized Third Normal Form (3NF) relational database, the data should be structurally separated into distinct, connected entities to eliminate redundancy and maintain integrity. At the foundation of this architecture is the Users entity, which serves as the core profile storing static personal details. This includes a unique primary key, the user's full name, a strictly unique constraint on the Malaysian IC number, and standard contact details like date of birth, gender, email, and phone number. Instead of bundling location data directly into the user profile, a separate Addresses entity should be established. Linked back to the user via a foreign key, this table will independently store the raw street address alongside the parsed postcode, city, and state, ensuring that if a user updates their location or has multiple addresses over time, the core profile remains untouched.

To manage the actual flow of services, an Applications entity acts as the transactional hub. Every time a user initiates a form-filling session, a new record is generated here, capturing the specific portal type—such as PTPTN, EPF, or LHDN—along with a timestamp and a foreign key linking back to the user. Running parallel to this is the Documents entity, which tracks the physical files processed by your Guardian agent. This table links directly to the relevant application and stores crucial file metadata, including the original filename, the file category, and the exact server paths for both the raw upload and the protected, watermarked output.

Finally, to accommodate the highly specific requirements of each government service without creating a massive, sparsely populated table full of null values, the architecture utilizes portal-specific detail tables. Each of these specialized tables uses the application ID as both a primary key and a foreign key. The PTPTN_Details entity captures education-centric data like parent income, degree name, and university. Similarly, the EPF_Details entity is dedicated to employment and withdrawal specifics, storing the EPF number, employer name, withdrawal type, and banking information. The LHDN_Details entity specifically isolates tax-related fields like the income tax reference number. This modular approach ensures that your database remains clean, highly scalable, and structurally sound as you expand the system to support additional government portals in the future.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	Features	Description
1	Conversational Document Ingestion (The Concierge)	Users can upload physical documents (e.g., EA Form, NRIC) via a chat interface. The system leverages the Z AI Model to perform OCR, understand the semantic context, and extract required data points into a strict JSON schema.
2	Client-Side Security & Encryption (The Guardian)	Before any processing, uploaded images are visually watermarked locally (e.g., "FOR LHDN USE ONLY"). All extracted JSON data is encrypted using AES-256 and stored securely in the browser's IndexedDB, ensuring zero remote data retention.
3	Autonomous Portal Autofill (The Executor)	A dedicated browser extension detects when the user navigates to a target government URL (e.g., LHDN MyTax). It decrypts the local data and uses heuristic DOM mapping to automatically populate the web form fields.
4	Human-in-the-Loop Verification	To ensure legal liability and accuracy, the Executor extension is programmatically blocked from submitting the form. It halts the process and prompts the user to manually review the populated fields and click the final "Submit" button.

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g.)
Security & Privacy	<i>Sensitive user data (e.g., NRIC, EA forms, income figures) must never be stored in plaintext on any external server to comply with Malaysian PDPA laws.</i>	<i>Implementation of a local-first architecture. All parsed JSON data is encrypted client-side using AES-256 via the Web Crypto API before being saved to the browser's IndexedDB. The Z AI Model API operates on a strict zero-retention policy.</i>
Scalability	<i>The system must be able to handle extreme spikes in user activity, specifically during the Malaysian tax filing season (March - May), without backend bottlenecks.</i>	<i>Because the system relies on a decentralized, client-side execution model, database reads/writes are handled by the user's own browser (IndexedDB). The Concierge PWA is statically hosted on a global CDN (e.g., Vercel) to absorb high frontend traffic seamlessly.</i>
Reliability & Accuracy	<i>The AI data extraction must have near-perfect accuracy for financial and identification data; it must absolutely not hallucinate missing values.</i>	<i>Use of "Zero-Imputation" prompt engineering and strict JSON schema validation. If the Z AI Model's confidence score drops, or a field is unreadable, it is forced to return null, which programmatically triggers the Concierge to ask the user for manual input.</i>
Token Latency	<i>The Z AI Model must process document OCR and return the structured JSON response within 4-6 seconds to maintain a fluid conversational experience.</i>	<i>Client-side image compression (restricting uploads to under 5MB) before API transmission. The UI implements asynchronous processing with "typing..." indicators to manage user expectations while awaiting the API response.</i>
Cost Efficiency	<i>The average token inference cost per user session must remain strictly bounded to keep the MVP viable at scale.</i>	<i>Context Window Pruning: The system only sends the immediate document and the last 3 chat interactions to the Z AI Model. Additionally, the actual form-filling relies entirely on zero-cost DOM mapping by the Executor Extension, reserving costly LLM calls only for the initial data extraction step.</i>

3.3. Out of Scope / Future Enhancements

a. Direct Payment Processing: Integrating payment gateways (e.g., FPX, DuitNow) to allow users to directly pay their LHDN taxes, PTPTN loans, or JPJ summons from within the Concierge interface.

b. Direct Government API Integrations: Bypassing the browser extension's DOM manipulation entirely by establishing official backend-to-backend API pipelines with government agencies (requires official partnerships).

c. Proactive Bureaucracy Alerts: A notification engine that tracks user deadlines and sends proactive alerts (e.g., "Your road tax expires in 14 days" or "LHDN tax filing season opens tomorrow").

d. Cross-Device Zero-Knowledge Sync: Allowing users to seamlessly sync their encrypted local data (IndexedDB) between their mobile phone and desktop browser using a secure, zero-knowledge cloud relay.

e. Corporate & Business Filings (B2B): Expanding the AI's semantic mapping capabilities to handle complex corporate tax filings (SME/Enterprise) rather than just individual citizen documents.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

Grayscale Rollout: Any updates to the **Z AI Model's prompt instructions** (e.g., adding support for a new tax document type) or the **Executor Extension's heuristic DOM mapping logic** will initially be deployed to just 5% of active users. The system will monitor the JSON schema validation success rate and the frequency of user manual corrections. If the error rate remains below the threshold, the rollout will gradually scale to 25%, 50%, and then 100%.

A/B Testing: We will conduct A/B testing on the **Concierge PWA** interface to determine which conversational prompts (Variant A: direct document upload vs. Variant B: step-by-step chat wizard) yield faster completion times and higher user confidence before they hit the final "Submit" button.

4.1.2. Emergency Rollback (ER) & Golden Release:

Emergency Strategy: If system monitoring detects a spike in DOM injection failures or if the Z AI Model begins failing schema validations (returning malformed JSON), an Emergency Rollback (ER) is automatically triggered.

Golden Release: The CI/CD pipelines (via Vercel for the PWA and the Chrome Web Store deployment scripts for the Extension) will instantly revert to the **Golden Release**—the last known stable version with a 99%+ successful mapping rate. During an ER, the Concierge chat will display a fallback message advising users to temporarily rely on manual data entry while the AI mappers are recalibrated.

4.1.3. **Priority Matrix:**

Priority Level	Trigger Conditions	Required Actions
P1 – Critical (Security / Data Integrity)	• Encryption Failure: Guardian agent fails to encrypt JSON, attempting to write plaintext to IndexedDB. • AI Hallucination Spike: Z AI Model fails the strict JSON schema validation (e.g., imputing fake data) in > 2% of transactions.	Auto-trigger Emergency Rollback (ER) to the Golden Release. Instantly halt all Executor extension background scripts to prevent bad data submission. Force active users to re-authenticate.
P2 - High (Core Feature Breakage)	• DOM Injection Failure: The Executor extension fails to map and inject data on a target portal (e.g., MyTax) with an error rate > 5%, usually due to unannounced UI updates by the government agency.	Temporarily disable the autofill capability for the specific affected portal via a remote feature flag. The Concierge UI will notify users to manually copy-paste their data until a hotfix is deployed for the heuristic mapping logic.
P3 - Medium (Performance Degradation)	• High Token Latency: The Z AI Model OCR and extraction process takes longer than 8 seconds for the 95th percentile (p95) of users. • Client-Side Throttling: The browser struggles to compress the uploaded images within the 5MB limit.	Trigger an automated alert to the development team to investigate API throttling or client-side memory leaks. Display a non-blocking warning in the Concierge chat: "Processing is taking longer than usual due to high traffic."

P4 - Low (Minor UI/UX Bugs)	• Minor visual glitches in the Concierge PWA (e.g., misaligned chat bubbles). • Typos in the non-critical fallback error messages.	Log the issue in the development backlog to be addressed and bundled into the next scheduled bi-weekly sprint deployment. No emergency actions are taken.

4.2. Monitoring:

The monitoring plan for GovEase AI focuses on ensuring the local backend server, the third-party AI integration, and the client extension are functioning correctly.

Backend Server Health Polling: * Plan: The Flask application includes a dedicated health check endpoint (`app.route("/health")`). A monitoring daemon or script will periodically ping `http://127.0.0.1:5000/health`.

Trigger: If the endpoint fails to return an HTTP 200 OK status with the `{"status": "healthy"}` payload, or if the connection times out, it indicates the local server has crashed. An alert will prompt the user to restart the `app.py` script.

API Integration & Extraction Monitoring:

Plan: The system heavily relies on the `api.ilmu.ai/anthropic` endpoint in `concierge_agent.py` for OCR and data extraction. We will monitor the response times and HTTP status codes of these external calls.

Trigger: If the API returns 401 Unauthorized (e.g., expired `ILMU_API_KEY`), 429 Too Many Requests, or 500 Internal Server Error, the backend's `try...except` blocks will catch the exception. The system will trigger an error state, logging `traceback.print_exc()` to the console and returning an explicit `{"status": "error"}` payload to the UI, notifying the user of the AI service disruption.

Extension-to-Server Connection Monitoring:

Plan: The Chrome extension (`content.js` and `popup.js`) relies on `fetch("http://localhost:5000/get-data")`.

Trigger: If the `fetch` request fails (caught in the `.catch(err)` block), the system assumes the backend is unreachable. The popup UI will immediately trigger a visual error state (✖ Cannot connect to server) advising the user to ensure Flask is running.

4.3. Assumptions:

Local Server Operation: It is assumed that users have Python installed and can successfully run the Flask backend (app.py) on localhost:5000 prior to interacting with the web interface or the Chrome extension.

Stable Network Connectivity: Users are assumed to have a stable internet connection. While the server runs locally, the core extraction engine requires internet access to communicate with the ILMU GLM-5.1 Vision model.

Browser Compatibility: It is assumed the user is utilizing a Chromium-based browser (like Google Chrome or Microsoft Edge) that fully supports Manifest V3 extensions, which is required for manifest.json and the chrome.storage.local APIs.

Static Government Portal DOM Structures: A critical assumption for the auto-fill functionality (content.js) is that the target government portals (PTPTN, EPF, LHDN) maintain relatively stable HTML structures. It is assumed that the form id, name, placeholder, or aria-label attributes continue to match the keywords predefined in the FIELD_MAP dictionary (e.g., "nama penuh", "no_kad").

Valid Document Uploads: It is assumed that users will upload legible, standard Malaysian Identity Cards (MyKad) or relevant documents in supported formats (.jpg, .jpeg, .png) to ensure the extract_document_data function and the guardian_agent watermarking process do not fail.

4.4. External Dependencies:

Tools	Purpose	Risks
Z AI Model API	Acts as the core LLM reasoning engine. It handles OCR for uploaded documents (like EA forms), semantic parsing of unstructured text, and generates the strict JSON output required for local storage.	High Risk: If the API experiences rate limitations or downtime, the core extraction feature fails. Mitigation: Implement a graceful degradation state where the Concierge PWA informs the user of the outage and provides a manual data-entry form as a fallback.
Vercel (or similar CDN)	Hosts the static assets for the Concierge Progressive Web App (PWA), ensuring fast, cross-platform access for both mobile and desktop users without app store deployment.	Medium Risk: CDN localized outages could prevent new users from loading the application. Mitigation: Implement robust PWA Service Workers so that returning users can still load the cached

		<i>application UI even during network instability.</i>
<i>Browser Extension Ecosystem (Manifest V3)</i>	<i>The runtime environment and distribution network (e.g., Chrome Web Store) required for the Executor Extension to operate and interact with external DOMs.</i>	High Risk: Sudden policy changes to Manifest V3 regarding content script injection or DOM access could temporarily disable the autofill capability. Mitigation: Strict adherence to standard Web APIs, minimizing required permissions, and keeping the content script logic highly modular for rapid compliance updates.
<i>Target Gov Portals (e.g., MyTax, PTPTN)</i>	<i>The external destination websites where the Executor Extension injects the decrypted user data.</i>	Medium Risk: Unannounced structural changes to the website's HTML layout. Mitigation: Utilizing AI-driven heuristic mapping (finding fields by label context) rather than relying on brittle, hardcoded HTML IDs.

5. Project Management & Team Contributions

Day 1:Align

Yit Cheng and Jun Xian:Setup GLM API,test OCR on sample IC image

Pei Feng:Draft PRD

Yu Xian: Research LHDN + PTPTN pain points

Xing You:Write script outline

Day 2:Build

Yit Cheng:Concierge chat + GLM OCR parsing working

Jun Xian: Chat UI + Chrome extension skeleton

Pei Feng:Finish PRD + sys diagram

Day 3:Connect

Yit Cheng:Guardian watermark agent done

Jun Xian:Extensions fills form fields

Yu Xian:First 5 QA test cases

Xing You:Finalise video script with Pei Feng

Day 4:Polish

Yit Cheng + Jun Xian:Fix bugs from Yu Xian's testing.Full end-to-end demo run.

Xing You:Finish QA doc + pitch deck.

Pei Feng:Record first video draft

Day 5:

Final video recording

Check all 6 deliverables